


# Universidad Mariano Gálvez de Guatemala.

ENTREGABLE FINAL

CURSO: ANÁLISIS DE SISTEMAS II

**DYLAN ALBERTO ROLDAN PEREZ** **9989-22-5223**
**FELIX FRANCISCO TOMA RODRIGUEZ** **99989-12-10925**
**DULCE CLAUDIA MICHELLE RAMIREZ RAC** **9989-23-15707**
**ANTHONY DE LEON** **9989-21-5704**

*1. INTRODUCCIÓN Y RESUMEN EJECUTIVO* ____ *3*

*2. REQUERIMIENTOS, MODELADO UML 2.5.1 Y GESTIÓN ÁGIL* ____ *4*

**2.1 Especificación de Requerimientos del Sistema ISO/IEC/IEEE 29148:2018** ____ **4**

2.1.1 Actores principales ____ 4

2.1.2 Requerimientos funcionales ____ 4

2.1.3 Requerimientos no funcionales ____ 5

2.1.4 Reglas de negocio ____ 5

**2.2 Diagramas UML 2.5.1** ____ **6**

2.2.1 Diagrama de Clases ____ 6

2.2.2 Diagrama de Secuencia ____ 6

2.2.3 Diagrama de Actividades con Calles de Responsabilidad (SWIMLANES) ____ 6

2.2.4 Diagrama de Paquetes ____ 6

**2.3 Fichas de Diseño de Componentes – SDD** ____ **7**

Componente: Autenticación ____ 7

Componente: Gestión de Contactos ____ 7

Componente: Gestión de Clientes ____ 7

Componente: Gestión de Proveedores ____ 7

**2.4 Gestión Ágil de Proyectos** ____ **8**

2.4.1 Marco de trabajo ____ 8

2.4.2 Rol del Product Owner / Analista ____ 8

2.4.3 Product Backlog con Historias de Usuario - LINK ____ 8

2.4.4 Definition of Ready (DoR) ____ 10

2.4.5 Definition of Done (DoD) ____ 11

*3. EVALUACIÓN ARQUITECTÓNICA, C4 Y PERSISTENCIA* ____ *12*

**3.1 Evaluación de Estilos Arquitectónicos y Escenarios de Calidad** ____ **12**

3.1.1 Comparación de estilos arquitectónicos ____ 12

**3.1.2** Matriz de Evaluación ISO/IEC 25010 ____ 13

3.1.3 Arquitectura seleccionada ____ 13

*3.2 Registros de Decisiones de Arquitectura (ADR - ISO/IEC/IEEE 42010:2011)* ____ *15*

**3.3 Documentación de la Arquitectura mediante el Modelo C4 (4 Niveles)** ____ **15**

**3.4 Arquitectura de Persistencia Políglota y Cache-Aside** ____ **15**

Distribución de responsabilidades ____ 15

Patrón Cache-Aside ____ 15

Política TTL ____ 16

Consistencia eventual ____ 16

**3.5 Aplicación de Patrones GoF y Análisis de Cohesión / Acoplamiento** ____ **16**

*4. DISEÑO DE APIS, IMPLEMENTACIÓN FUNCIONAL Y BDD* ____ *18*

**4.1 Filosofía API-First y Contrato OpenAPI 3.0.3** ____ **18**

**4.2 Estándar de Errores RFC 9110 y RFC 9457** ____ **19**

**4.3 Catálogo de Endpoints Funcionales Desarrollados** ____ **20**

**4.4 Especificación BDD y Casos de Prueba de Aceptación GHERKIN** ____ **21**
Vínculos ____ 21
***5. EXPERIENCIA DE USUARIO (UX/UI) Y MOBILE-FIRST*** ____ ***26***
**5.1 Diseño Centrado en el Usuario (ISO 9241-210) y Customer Journey Map** ____ **26**
**Customer Journey Map** ____ **26**
**5.2 Evaluación Heurística, ISO 9241-11 y WCAG 2.2** ____ **27**
**5.3 Estrategia Mobile-First y Atomic Design** ____ **28**
***6. ESTRATEGIA DE CALIDAD Y PLAN SQA*** ____ ***29***
**6.1 Plan de Aseguramiento de Calidad SQA (IEEE 730-2014) y Enfoque Shift-Left** ____ **29**
**6.2 Pirámide de Pruebas Automatizadas (Unitarias, Integración, E2E)** ____ **30**
**6.3 Clasificación y Flujo de Gestión de Defectos (IEEE 1044-2009)** ____ **31**
***7. MATRIZ DE TRAZABILIDAD INTEGRAL*** ____ ***32***
***PRUEBAS*** ____ ***34***
***8. CONCLUSIONES*** ____ ***38***
***9. REFERENCIAS*** ____ ***39***

# 1. INTRODUCCIÓN Y RESUMEN EJECUTIVO

El presente proyecto tiene como propósito analizar, diseñar y desarrollar un módulo CRM orientado a la gestión de contactos, clientes y proveedores, aplicando estándares internacionales de ingeniería de software, arquitectura, calidad y experiencia de usuario.

El alcance comprende la especificación formal de requerimientos, modelado UML, definición de la arquitectura del sistema, diseño de APIs bajo el enfoque API-First, implementación funcional del backend, pruebas BDD, diseño UX/UI, accesibilidad y establecimiento de una estrategia de aseguramiento de calidad.

El desarrollo se organiza mediante una metodología de diez pasos que integra la especificación de requerimientos, evaluación arquitectónica, diseño de APIs, persistencia de datos, modelado C4, gestión ágil, pruebas BDD, diseño centrado en el usuario, estrategia Mobile-First y aseguramiento de calidad. Estos elementos permiten mantener trazabilidad entre las necesidades del sistema, su arquitectura, implementación y validación.



# 2. REQUERIMIENTOS, MODELADO UML 2.5.1 Y GESTIÓN ÁGIL

## 2.1 Especificación de Requerimientos del Sistema ISO/IEC/IEEE 29148:2018

La especificación de requerimientos define las funciones, restricciones y características de calidad que debe cumplir el módulo CRM. Los requerimientos se redactan utilizando una estructura formal basada en **Sujeto + Verbo de obligación + Acción + Condición**.

### 2.1.1 Actores principales

- **Usuario:** gestiona contactos, clientes y proveedores.
- **Administrador:** administra información y posee permisos adicionales.
- **Sistema CRM:** procesa las operaciones solicitadas.
- **Base de datos:** almacena la información persistente del sistema.

### 2.1.2 Requerimientos funcionales

| ID | Requerimiento |
|---|---|
| **REQ-FN-001** | El sistema **deberá autenticar** al usuario mediante sus credenciales antes de permitir el acceso a las funcionalidades protegidas. |
| **REQ-FN-002** | El sistema **deberá permitir registrar** nuevos contactos cuando se proporcionen los datos obligatorios válidos. |
| **REQ-FN-003** | El sistema **deberá permitir consultar** la información de los contactos registrados según los permisos del usuario autenticado. |
| **REQ-FN-004** | El sistema **deberá permitir actualizar** los datos de un contacto existente cuando el usuario posea los permisos correspondientes. |
| **REQ-FN-005** | El sistema **deberá permitir eliminar** un contacto existente cuando la operación sea solicitada por un usuario autorizado. |
| **REQ-FN-006** | El sistema **deberá permitir clasificar** un contacto como cliente o proveedor según la información registrada. |
| **REQ-FN-007** | El sistema **deberá permitir buscar y filtrar** contactos utilizando criterios definidos por el usuario. |
| **REQ-FN-008** | El sistema **deberá devolver respuestas estructuradas mediante la API** para las operaciones realizadas por las aplicaciones cliente. |

### 2.1.3 Requerimientos no funcionales

| ID | Requerimiento |
|---|---|
| **REQ-NF-001** | El sistema **deberá proteger** los endpoints restringidos mediante autenticación basada en JWT. |
| **REQ-NF-002** | El sistema **deberá validar** los datos recibidos antes de almacenarlos o procesarlos. |
| **REQ-NF-003** | El sistema **deberá utilizar** respuestas HTTP estandarizadas para indicar el resultado de cada operación. |
| **REQ-NF-004** | El sistema **deberá mantener** una estructura modular que permita modificar sus componentes sin afectar innecesariamente otros módulos. |
| **REQ-NF-005** | El sistema **deberá proporcionar** respuestas de error estructuradas que permitan identificar claramente la causa del fallo. |
| **REQ-NF-006** | El sistema **deberá utilizar mecanismos de caché** en las consultas definidas como de acceso frecuente para mejorar el desempeño. |

### 2.1.4 Reglas de negocio

| ID | Regla |
|---|---|
| **RN-001** | Todo contacto deberá poseer un identificador único. |
| **RN-002** | Un cliente o proveedor deberá estar asociado previamente a un contacto existente. |
| **RN-003** | Las operaciones protegidas solo podrán ejecutarse mediante un usuario autenticado. |
| **RN-004** | Los datos obligatorios deberán ser validados antes de registrar o actualizar información. |
| **RN-005** | Los privilegios administrativos deberán limitarse a usuarios con el rol correspondiente. |

Los identificadores definidos en esta sección serán utilizados posteriormente para establecer la trazabilidad con Historias de Usuario, diagramas UML y C4, endpoints OpenAPI, escenarios BDD y casos de prueba.

LINK

## 2.2 Diagramas UML 2.5.1

### 2.2.1 Diagrama de Clases

El diagrama de clases representa la estructura principal del módulo CRM y las relaciones existentes entre sus entidades.

Las clases principales son **Usuario, Contacto, Cliente y Proveedor**. Un usuario puede registrar múltiples contactos, mientras que cada contacto puede especializarse como cliente o proveedor.

Las relaciones incluyen cardinalidades y dependencias entre las entidades principales del sistema.

LINK

### 2.2.2 Diagrama de Secuencia

El diagrama de secuencia representa el flujo de interacción necesario para registrar un nuevo contacto en el sistema. Participan el usuario, la interfaz cliente, la API del CRM, el controlador, el modelo y la base de datos.

El flujo incluye la autenticación mediante token JWT, validación de datos, registro del contacto y devolución de la respuesta correspondiente.

LINK

### 2.2.3 Diagrama de Actividades con Calles de Responsabilidad (SWIMLANES)

El diagrama de actividades representa el proceso de creación de un contacto, distribuyendo las responsabilidades entre el usuario, la aplicación cliente, la API y la base de datos.

Este modelado permite identificar claramente qué participante ejecuta cada actividad y cómo se manejan las validaciones y posibles errores durante el proceso.

LINK

### 2.2.4 Diagrama de Paquetes

El diagrama de paquetes representa la organización lógica del módulo CRM y la separación de responsabilidades entre autenticación, gestión de contactos, gestión de clientes y proveedores, acceso a datos y servicios de infraestructura. Esta organización favorece la modularidad, mantenibilidad y bajo acoplamiento entre componentes.

LINK

## 2.3 Fichas de Diseño de Componentes – SDD

**Estándar aplicado: IEEE 1016-2009**

Las fichas SDD describen la responsabilidad, interfaces, dependencias y manejo de excepciones de los principales componentes del módulo CRM.

### Componente: Autenticación

| Elemento | Descripción |
|---|---|
| **Responsabilidad** | Validar credenciales y generar tokens JWT. |
| **Entradas** | Correo electrónico y contraseña. |
| **Salidas** | Token JWT y datos básicos del usuario. |
| **Interfaces** | POST /api/v1/auth/login |
| **Dependencias** | Modelo Usuario, servicio JWT y base de datos. |
| **Excepciones** | Credenciales inválidas, usuario inexistente o token inválido. |

### Componente: Gestión de Contactos

| Elemento | Descripción |
|---|---|
| **Responsabilidad** | Crear, consultar, actualizar, eliminar y buscar contactos. |
| **Entradas** | Datos del contacto y parámetros de consulta. |
| **Salidas** | Información de contactos y resultados de las operaciones. |
| **Interfaces** | Endpoints REST de contactos. |
| **Dependencias** | ContactoController, modelo Contacto, persistencia y caché. |
| **Excepciones** | Datos inválidos, contacto inexistente o acceso no autorizado. |

### Componente: Gestión de Clientes

| Elemento | Descripción |
|---|---|
| **Responsabilidad** | Gestionar la información específica de clientes asociados a contactos. |
| **Entradas** | Identificador del contacto y código de cliente. |
| **Salidas** | Información del cliente. |
| **Interfaces** | Servicios internos y endpoints relacionados con clientes. |
| **Dependencias** | Modelo Cliente y modelo Contacto. |
| **Excepciones** | Contacto inexistente, cliente duplicado o datos inválidos. |

### Componente: Gestión de Proveedores

| Elemento | Descripción |
|---|---|
| **Responsabilidad** | Gestionar proveedores asociados a los contactos registrados. |
| **Entradas** | Identificador del contacto y código de proveedor. |
| **Salidas** | Información del proveedor. |

| | |
|---|---|
| **Interfaces** | Servicios internos y endpoints relacionados con proveedores. |
| **Dependencias** | Modelo Proveedor y modelo Contacto. |
| **Excepciones** | Contacto inexistente, proveedor duplicado o datos inválidos. |

Estas fichas permiten documentar las responsabilidades de cada módulo y facilitan la correspondencia entre el diseño, la arquitectura y la implementación del sistema.

LINK

## 2.4 Gestión Ágil de Proyectos

### 2.4.1 Marco de trabajo

Para la gestión del proyecto se utiliza un enfoque ágil basado principalmente en **Scrum**, complementado con prácticas de **Kanban** para visualizar el flujo de trabajo y el estado de las tareas.

Scrum permite organizar el desarrollo mediante iteraciones, priorización del Product Backlog y entregas incrementales. Kanban se utiliza como apoyo visual para controlar actividades pendientes, en progreso y finalizadas.

### 2.4.2 Rol del Product Owner / Analista

El Product Owner es responsable de representar las necesidades del sistema y priorizar el trabajo del equipo.

Sus principales responsabilidades son:

- Definir y priorizar el Product Backlog.
- Redactar y validar Historias de Usuario.
- Establecer criterios de aceptación.
- Aclarar requerimientos funcionales y reglas de negocio.
- Verificar que cada incremento aporte valor al sistema.
- Mantener la trazabilidad entre requerimientos, Historias de Usuario y funcionalidades implementadas.

### 2.4.3 Product Backlog con Historias de Usuario - LINK

El Product Backlog contiene las funcionalidades principales del módulo CRM. Las Historias de Usuario se estructuran bajo el enfoque **3C: Card, Conversation y Confirmation**, y se validan utilizando los principios **INVEST**.

| ID | Historia de Usuario | Prioridad |
|---|---|---|

| | | |
|---|---|---|
| HU-01 | Como usuario, quiero iniciar sesión para acceder de forma segura al sistema. | Alta |
| HU-02 | Como usuario, quiero registrar contactos para almacenar su información en el CRM. | Alta |
| HU-03 | Como usuario, quiero consultar contactos para visualizar la información registrada. | Alta |
| HU-04 | Como usuario, quiero actualizar contactos para mantener sus datos vigentes. | Alta |
| HU-05 | Como usuario autorizado, quiero eliminar contactos para retirar registros que ya no sean necesarios. | Media |
| HU-06 | Como usuario, quiero clasificar contactos como clientes o proveedores para organizar mejor la información. | Alta |
| HU-07 | Como usuario, quiero buscar y filtrar contactos para localizar información rápidamente. | Media |
| HU-08 | Como administrador, quiero gestionar permisos de acceso para controlar las operaciones disponibles para cada usuario. | Media |

Aplicación del modelo 3C

| ID | Card | Conversation | Confirmation |
|---|---|---|---|
| HU-01 | Como usuario, quiero iniciar sesión para acceder de forma segura al CRM. | Se utilizarán correo y contraseña. La cuenta debe estar activa y, si las credenciales son correctas, se generará un JWT. | Credenciales válidas → 200; inválidas → 401; datos con formato incorrecto → 422. |
| HU-02 | Como usuario, quiero registrar contactos para almacenar su información. | Se requieren tipo, nombre, apellido y teléfono. Se validarán los datos antes de almacenarlos. | Datos válidos → 201; datos inválidos → 422; el contacto queda registrado. |
| HU-03 | Como usuario, quiero consultar contactos para visualizar la información registrada. | La consulta requiere JWT y debe respetar los permisos del usuario. Puede utilizar Redis mediante Cache-Aside. | Token válido → 200; token inválido → 401; se devuelve únicamente información autorizada. |
| HU-04 | Como usuario, quiero actualizar contactos para mantener sus datos vigentes. | Solo podrán modificarse contactos existentes y autorizados. La actualización deberá invalidar la caché correspondiente. | Actualización correcta → 200; inexistente → 404; datos inválidos → 422. |
| HU-05 | Como usuario autorizado, quiero | Se realizará eliminación lógica para conservar el historial del registro. | Operación válida → 200; contacto inexistente → |

| ID | Card | Conversation | Confirmation |
|---|---|---|---|
| | desactivar contactos que ya no sean necesarios. | | 404; acceso no autorizado → 403. |
| HU-06 | Como usuario, quiero clasificar contactos como clientes o proveedores. | Un cliente requiere codigo_cliente; un proveedor requiere los datos específicos correspondientes. | La clasificación válida se almacena correctamente; información incompleta → 422. |
| HU-07 | Como usuario, quiero buscar y filtrar contactos para localizar información rápidamente. | Se podrá filtrar por datos como nombre, tipo y otros criterios disponibles en la API. | La búsqueda válida → 200; sin coincidencias → lista vacía sin producir error. |
| HU-08 | Como administrador, quiero gestionar permisos para controlar las operaciones disponibles. | Los permisos dependen del rol del usuario. Las operaciones restringidas deben comprobar autorización. | Administrador autorizado → operación permitida; usuario sin permisos → 403 Forbidden. |

Validación INVEST

Las Historias de Usuario fueron definidas procurando que sean:

- I – Independent: independientes en la medida de lo posible.
- N – Negotiable: susceptibles de ajuste durante el desarrollo.
- V – Valuable: aportan valor al usuario o al sistema.
- E – Estimable: pueden estimarse en esfuerzo.
- S – Small: poseen un alcance controlado.
- T – Testable: pueden comprobarse mediante criterios de aceptación y pruebas.

### 2.4.4 Definition of Ready (DoR)

Una Historia de Usuario se considera lista para iniciar cuando:

- Está redactada de forma clara.
- Tiene prioridad definida.
- Incluye criterios de aceptación.
- Sus dependencias están identificadas.
- Puede ser estimada por el equipo.
- No presenta dudas funcionales críticas.
- Cumple razonablemente con los principios INVEST.

### 2.4.5 Definition of Done (DoD)

Una Historia de Usuario se considera finalizada cuando:

- La funcionalidad fue implementada.
- El código fue probado correctamente.
- Los criterios de aceptación fueron cumplidos.
- Los endpoints relacionados funcionan según el contrato API.
- Las pruebas asociadas fueron ejecutadas satisfactoriamente.
- La documentación correspondiente fue actualizada.
- No existen defectos críticos pendientes.

Vinculos:
Diagramas
Backlog.md

`docs/requirements`

# 3. EVALUACIÓN ARQUITECTÓNICA, C4 Y PERSISTENCIA

## 3.1 Evaluación de Estilos Arquitectónicos y Escenarios de Calidad

Para definir la arquitectura del módulo CRM se analizaron diferentes estilos arquitectónicos considerando atributos de calidad establecidos por **ISO/IEC 25010**, principalmente mantenibilidad, desempeño y escalabilidad.

### 3.1.1 Comparación de estilos arquitectónicos

| **Estilo / Arquitectura** | **Ventajas** | **Desventajas** | **Riesgos** |
|---|---|---|---|
| Arquitectura en Capas | Separa responsabilidades y facilita mantenimiento. | Puede generar dependencias fuertes entre capas. | Acoplamiento excesivo si no se diseña correctamente. |
| Repositorio | Centraliza y organiza el acceso a datos. | Puede crear dependencia fuerte hacia la capa de persistencia. | El repositorio puede convertirse en un punto crítico. |
| Microservicios | Permite escalamiento y despliegue independiente. | Mayor complejidad de infraestructura y comunicación. | Fallos distribuidos y problemas de consistencia. |
| EDA | Favorece desacoplamiento y procesamiento asíncrono. | Mayor dificultad de seguimiento y depuración. | Eventos duplicados, perdidos o fuera de orden. |
| Monolítica Modular | Mantiene un único despliegue, pero separa claramente los módulos y responsabilidades. Es más sencilla de operar y mantener. | Los módulos siguen compartiendo el mismo proceso y despliegue. | Si no se respetan los límites entre módulos, puede convertirse en un monolito altamente acoplado. |

### 3.1.2 Matriz de Evaluación ISO/IEC 25010

| Arquitectura | Mantenibilidad | Desempeño | Escalabilidad | Seguridad | Disponibilidad | Complejidad |
|---|---|---|---|---|---|---|
| Capas | Alta | Alta | Media | Alta | Media | Baja |
| Repositorio | Alta | Alta | Media | Media | Media | Baja |
| Microservicios | Alta | Alta | Muy Alta | Alta | Alta | Muy Alta |
| EDA | Alta | Alta | Alta | Media-Alta | Alta | Alta |
| Monolítica Modular | Muy Alta | Alta | Media-Alta | Alta | Media-Alta | Media-Baja |

La matriz utiliza atributos de calidad relacionados con ISO/IEC 25010 para comparar las alternativas arquitectónicas. Para el alcance actual del módulo CRM se seleccionó la **Arquitectura Monolítica Modular**, debido a que proporciona alta mantenibilidad, buen desempeño y seguridad, manteniendo una complejidad operativa menor que microservicios o EDA.

### 3.1.3 Arquitectura seleccionada

Para el módulo CRM se seleccionó una **Arquitectura Monolítica Modular**.

La aplicación se ejecuta como una única solución desplegable, pero internamente se encuentra dividida en módulos con responsabilidades específicas, como:

- Autenticación.
- Gestión de contactos.
- Gestión de clientes.
- Gestión de proveedores.
- Persistencia.
- Caché.

Cada módulo mantiene una separación clara entre controladores, servicios, modelos y acceso a datos.

La arquitectura monolítica modular fue seleccionada porque el tamaño y alcance actual del CRM no justifican la complejidad operativa de una arquitectura basada completamente en microservicios.

Este enfoque permite conservar una estructura organizada y mantenible, reducir la complejidad de despliegue y facilitar las pruebas e integración de los componentes.

Además, la separación modular permite que determinados módulos puedan evolucionar posteriormente hacia servicios independientes si el crecimiento del sistema lo requiere.

### 3.1.4 Relación con los estilos evaluados

La solución incorpora conceptos de los estilos evaluados sin adoptar completamente cada uno:

- **Capas:** separación entre controladores, servicios, modelos y persistencia.
- **Repositorio:** abstracción del acceso a los datos.
- **Microservicios:** separación lógica de responsabilidades por módulos, sin realizar despliegues independientes.
- **EDA:** puede utilizarse de forma complementaria para operaciones asíncronas cuando sea necesario.

### 3.1.5 Escenarios de calidad

**Escenario 1 – Mantenibilidad**

Cuando se requiera modificar la lógica de gestión de contactos, los cambios deberán mantenerse principalmente dentro del módulo de contactos sin afectar innecesariamente otros módulos.

**Escenario 2 – Desempeño**

Cuando un usuario realice una consulta frecuente, el sistema deberá aprovechar mecanismos de caché mediante Redis para reducir consultas repetitivas hacia la persistencia principal.

**Escenario 3 – Escalabilidad**

Cuando aumente la demanda del sistema, la arquitectura modular deberá permitir identificar los componentes de mayor carga y facilitar su futura separación o escalamiento.

**Escenario 4 – Disponibilidad**

Cuando falle una dependencia secundaria, el sistema deberá controlar la excepción y evitar que el fallo provoque la interrupción completa de la aplicación.

**Escenario 5 – Seguridad**

Cuando un usuario intente acceder a un recurso protegido, el sistema deberá verificar su autenticación y autorización antes de permitir la operación.

### 3.1.6 Conclusión de la evaluación

La Arquitectura Monolítica Modular ofrece un equilibrio adecuado entre mantenibilidad, desempeño, simplicidad operativa y posibilidad de crecimiento para el CRM. Permite mantener los componentes organizados y desacoplados sin introducir prematuramente la complejidad asociada con sistemas distribuidos.

## 3.2 Registros de Decisiones de Arquitectura (ADR - ISO/IEC/IEEE 42010:2011)

LINK

## 3.3 Documentación de la Arquitectura mediante el Modelo C4 (4 Niveles)

LINK

### 3.4 Arquitectura de Persistencia Políglota y Cache-Aside

La arquitectura actual del CRM utiliza **MySQL** como fuente principal de información y **Redis** como mecanismo de caché. Como evolución arquitectónica se propone adoptar persistencia políglota, asignando cada tecnología al tipo de información para el cual resulta más adecuada.

#### Distribución de responsabilidades

| Tecnología | Responsabilidad |
|---|---|
| **PostgreSQL** | Persistencia relacional principal para usuarios, contactos, clientes, proveedores y operaciones transaccionales. |
| **MongoDB** | Información documental o con estructura flexible, como historiales, registros o metadatos. |
| **Redis** | Caché temporal para consultas frecuentes y reducción de accesos a la base de datos. |
| **Neo4j** | Relaciones complejas entre entidades cuando sea necesario realizar análisis basados en grafos. |

#### Patrón Cache-Aside

Para las consultas frecuentes se utilizará el patrón **Cache-Aside**:

1. La API consulta primero Redis.
2. Si el dato existe, se produce un **Cache Hit** y se devuelve inmediatamente.
3. Si el dato no existe, se produce un **Cache Miss**.
4. La API consulta la base de datos principal.
5. El resultado se almacena en Redis con un TTL.
6. Finalmente se devuelve la información al cliente.

Cuando la información sea modificada o eliminada, la clave correspondiente de Redis deberá invalidarse para evitar datos obsoletos.

### Política TTL

El tiempo de expiración dependerá del tipo de información. Los datos consultados frecuentemente podrán mantenerse temporalmente en caché, mientras que información sensible a cambios utilizará períodos menores.

### Consistencia eventual

MongoDB, Redis y Neo4j podrán contener representaciones derivadas de información almacenada en la fuente principal. Por esta razón, algunos cambios podrán propagarse de forma asíncrona.

La consistencia transaccional deberá mantenerse en PostgreSQL, mientras que los demás motores podrán aceptar **consistencia eventual** cuando el caso de uso lo permita.

Ante fallos de sincronización, el sistema deberá conservar la fuente principal como referencia y permitir reprocesar las actualizaciones pendientes.

## 3.5 Aplicación de Patrones GoF y Análisis de Cohesión / Acoplamiento

El diseño del CRM utiliza diferentes patrones para separar responsabilidades, reducir dependencias y facilitar futuras modificaciones.

**Patrones aplicados**

| Patrón | Aplicación en el CRM |
|---|---|

| | |
|---|---|
| **Strategy** | Permite seleccionar diferentes estrategias de búsqueda, filtrado o procesamiento sin modificar el controlador principal. |
| **Repository** | Centraliza el acceso a los datos y evita que los servicios dependan directamente del ORM o de consultas SQL. |
| **Gateway** | Encapsula la comunicación con servicios externos, como correo electrónico o APIs externas. |
| **Observer** | Permite reaccionar ante eventos como creación o actualización de un contacto, por ejemplo para generar notificaciones o registros. |
| **Circuit Breaker** | Controla fallos repetitivos al consumir servicios externos, evitando llamadas continuas a un servicio que no se encuentra disponible. |

### Ejemplo de organización

El flujo principal de acceso a datos será:

Controller → Service → Repository → Modelo → Base de Datos

Para servicios externos:

Service → Gateway → Servicio Externo

### Cohesión

La arquitectura busca una **alta cohesión**, manteniendo dentro de cada módulo únicamente las responsabilidades relacionadas con su función.

Por ejemplo, el módulo de Contactos concentra la creación, consulta, actualización y eliminación de contactos, mientras que autenticación mantiene su lógica separada.

### Acoplamiento

Se busca mantener un **bajo acoplamiento** mediante servicios, repositorios e interfaces.

Los controladores no acceden directamente a la base de datos, sino que delegan la lógica a los servicios y repositorios. Esto permite modificar una implementación sin afectar innecesariamente otros componentes.

#### Beneficios

La aplicación de estos patrones permite:

- Facilitar el mantenimiento.
- Mejorar la capacidad de prueba.
- Reducir dependencias directas.
- Sustituir implementaciones con menor impacto.

- Preparar el sistema para futuras integraciones y crecimiento.

# 4. DISEÑO DE APIS, IMPLEMENTACIÓN FUNCIONAL Y BDD

## 4.1 Filosofía API-First y Contrato OpenAPI 3.0.3

El desarrollo del módulo CRM adopta el enfoque **API-First**, estableciendo primero el contrato de comunicación de la API antes de completar su implementación interna.

El contrato se documenta mediante **OpenAPI 3.0.3** y define rutas, métodos HTTP, parámetros, cuerpos de solicitud, esquemas de datos, autenticación y respuestas esperadas.

La API utiliza el prefijo:

/api/v1

**Recursos principales**

La API expone recursos relacionados con autenticación, contactos, historial de cambios y relaciones entre entidades.

| Recurso | Función | Tecnología principal |
|---|---|---|
| Autenticación | Validación de usuarios y generación JWT | PostgreSQL |
| Contactos | Gestión transaccional de contactos | PostgreSQL |
| Consultas frecuentes | Optimización de lecturas | Redis |
| Historial | Registro documental de cambios | MongoDB |
| Relaciones | Consulta de conexiones entre contactos | Neo4j |

**Persistencia políglota**

El contrato API mantiene transparente para el consumidor la tecnología utilizada internamente. El cliente realiza solicitudes HTTP normales, mientras que la capa de servicios selecciona el mecanismo de persistencia correspondiente.

Por ejemplo, una consulta de contactos puede resolverse inicialmente mediante Redis. En caso de un Cache Miss, la información será obtenida desde PostgreSQL y posteriormente almacenada temporalmente en Redis.

Las modificaciones realizadas sobre los contactos podrán generar registros históricos almacenados en MongoDB, mientras que Neo4j permitirá consultar relaciones entre entidades.

**Seguridad**

La API utiliza autenticación mediante **Bearer JWT**. El endpoint de inicio de sesión es público, mientras que los recursos de contactos, historial y relaciones requieren un token válido.

**Manejo estandarizado de errores**

Las respuestas de error se definirán utilizando los códigos HTTP establecidos por RFC 9110 y el formato **Problem Details** definido por RFC 9457.

El tipo de contenido utilizado será:

application/problem+json

Esto proporciona una estructura uniforme para errores de autenticación, autorización, validación, recursos inexistentes y errores internos.

**Integración conceptual con GraphQL y gRPC**

La API principal del CRM se mantiene bajo el estilo REST y se documenta mediante OpenAPI 3.0.3. Sin embargo, la arquitectura permite considerar otras tecnologías de integración según futuras necesidades.

**GraphQL** podría utilizarse para consultas complejas donde el cliente necesite seleccionar únicamente determinados campos o combinar información de contactos, historial y relaciones en una sola solicitud.

**gRPC** podría utilizarse para comunicación interna entre servicios si, en una futura evolución, algunos módulos del Monolito Modular fueran separados como servicios independientes. Su uso permitiría comunicaciones eficientes mediante contratos fuertemente tipados.

Estas tecnologías se consideran extensiones futuras y **no sustituyen actualmente la API REST implementada**.

**Contrato**

La especificación completa de la API se encuentra disponible en:

docs/api/openapi.yaml

## 4.2 Estándar de Errores RFC 9110 y RFC 9457

La API del módulo CRM utiliza códigos de estado HTTP de acuerdo con **RFC 9110** y estandariza las respuestas de error mediante **RFC 9457 – Problem Details for HTTP APIs**.

Todas las respuestas de error utilizan el tipo de contenido: application/problem+json

```
{
  "type": "https://api.crm.local/problems/recurso-no-encontrado",
  "title": "Recurso no encontrado",
  "status": 404,
  "detail": "El contacto solicitado no existe.",
  "instance": "/api/v1/contactos/999"
}
```

Los principales códigos utilizados son:

| Código | Significado | Uso en el CRM |
|---|---|---|
| 400 | Bad Request | Solicitud mal formada |
| 401 | Unauthorized | Token inexistente, inválido o expirado |
| 403 | Forbidden | Usuario autenticado sin permisos |
| 404 | Not Found | Recurso inexistente |
| 422 | Unprocessable Content | Error de validación |
| 500 | Internal Server Error | Error inesperado del servidor |

Para errores de validación se agrega el campo `errors`:

```
{
  "type": "https://api.crm.local/problems/error-validacion",
  "title": "Error de validación",
  "status": 422,
  "detail": "Uno o más campos contienen valores inválidos.",
  "instance": "/api/v1/contactos",
  "errors": {
    "nombre": [
      "El nombre es obligatorio."
    ]
  }
}
```

## 4.3 Catálogo de Endpoints Funcionales Desarrollados

Para demostrar el funcionamiento de la API y su integración con la persistencia políglota se seleccionan cinco endpoints principales. Estos permiten cubrir autenticación, operaciones transaccionales, caché, almacenamiento documental y relaciones mediante grafos.

| Método HTTP | Endpoint | Descripción | Persistencia / Caché | Estado |
|---|---|---|---|---|
| POST | /api/v1/auth/login | Autentica al usuario y genera un token JWT. | PostgreSQL | Completado |
| GET | /api/v1/contactos | Consulta contactos utilizando Cache-Aside. | Redis / PostgreSQL | Completado |
| POST | /api/v1/contactos | Registra un contacto y genera su evento histórico. | PostgreSQL / MongoDB / Redis | Completado |
| GET | /api/v1/contactos/{id}/ | Consulta el historial de cambios del contacto. | PostgreSQL / MongoDB / Redis | Completado |

El endpoint GET /contactos aplicará **Cache-Aside**: primero consultará Redis y, en caso de no encontrar información, obtendrá los datos desde PostgreSQL y almacenará temporalmente el resultado en caché.

El endpoint POST /contactos utilizará PostgreSQL como fuente transaccional principal, invalidará la caché relacionada en Redis y generará un registro histórico en MongoDB.

## 4.4 Especificación BDD y Casos de Prueba de Aceptación GHERKIN

La validación funcional del CRM utiliza **BDD (Behavior Driven Development)** mediante archivos Gherkin, siguiendo la estructura **Dado / Cuando / Entonces**.

Los escenarios representan comportamientos verificables de la API y posteriormente se relacionarán con los casos de prueba documentados bajo **ISO/IEC/IEEE 29119-3:2021**.

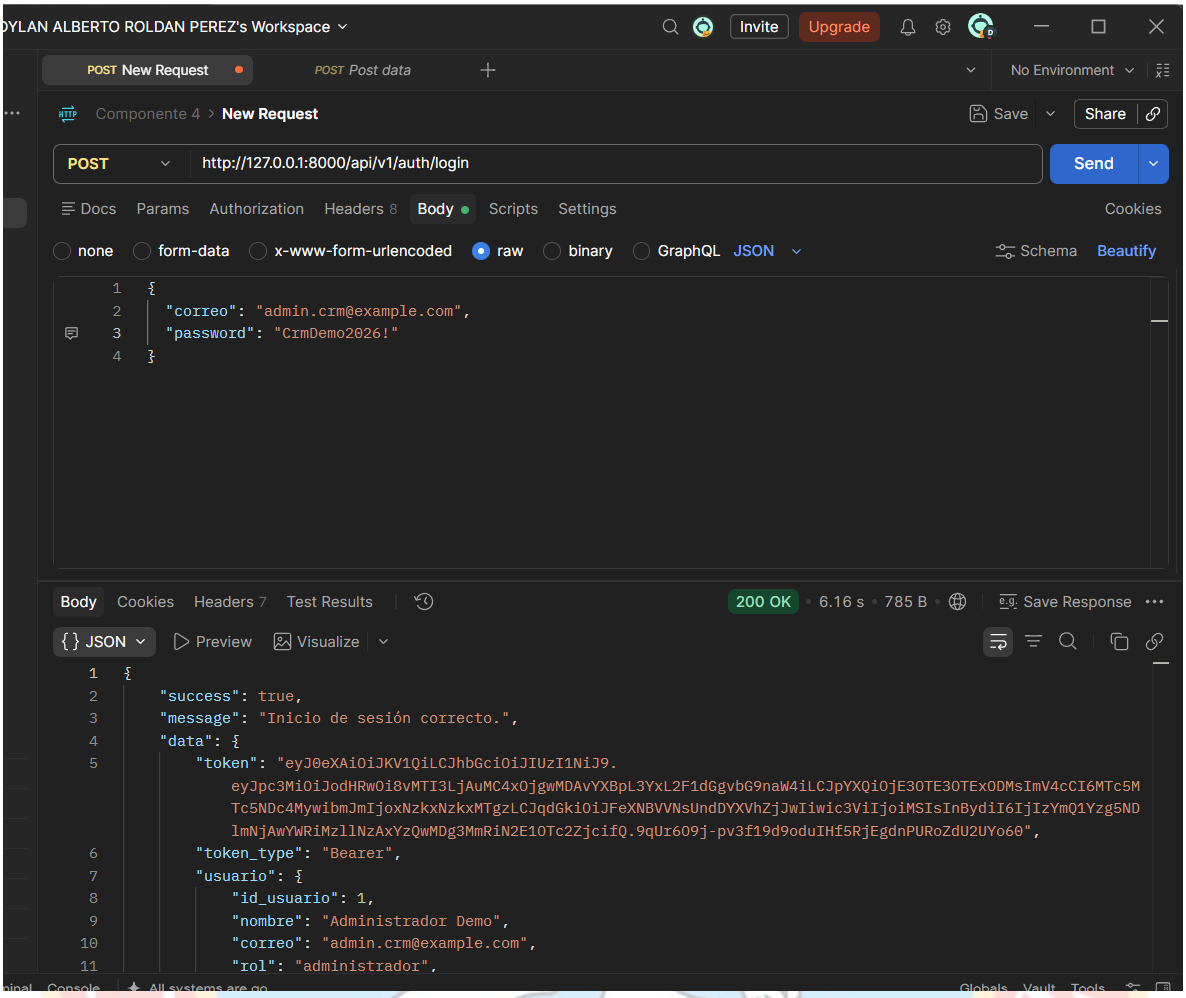
Se incluyen escenarios parametrizados mediante **Scenario Outline**, permitiendo ejecutar un mismo comportamiento con diferentes datos de prueba.

### Vínculos

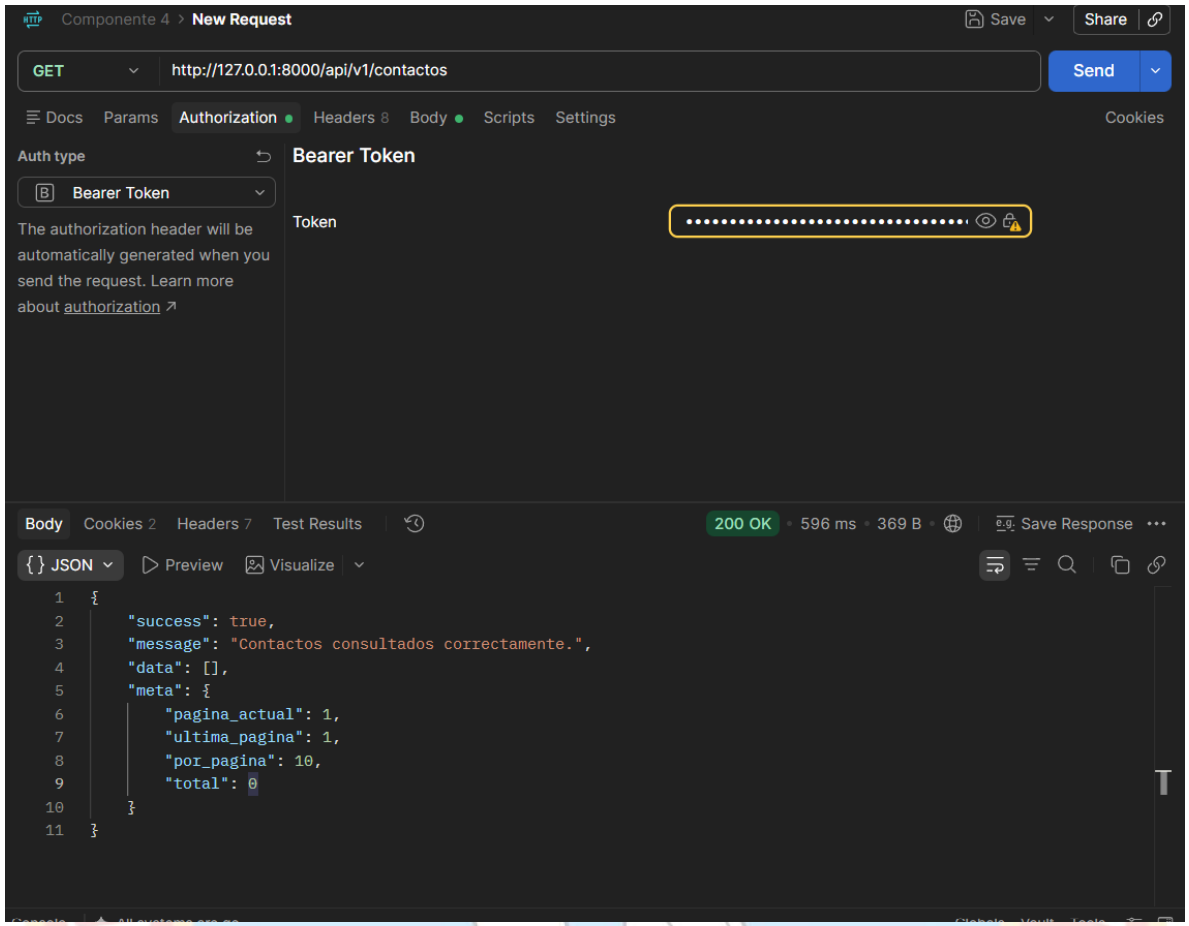
Features/ LINK

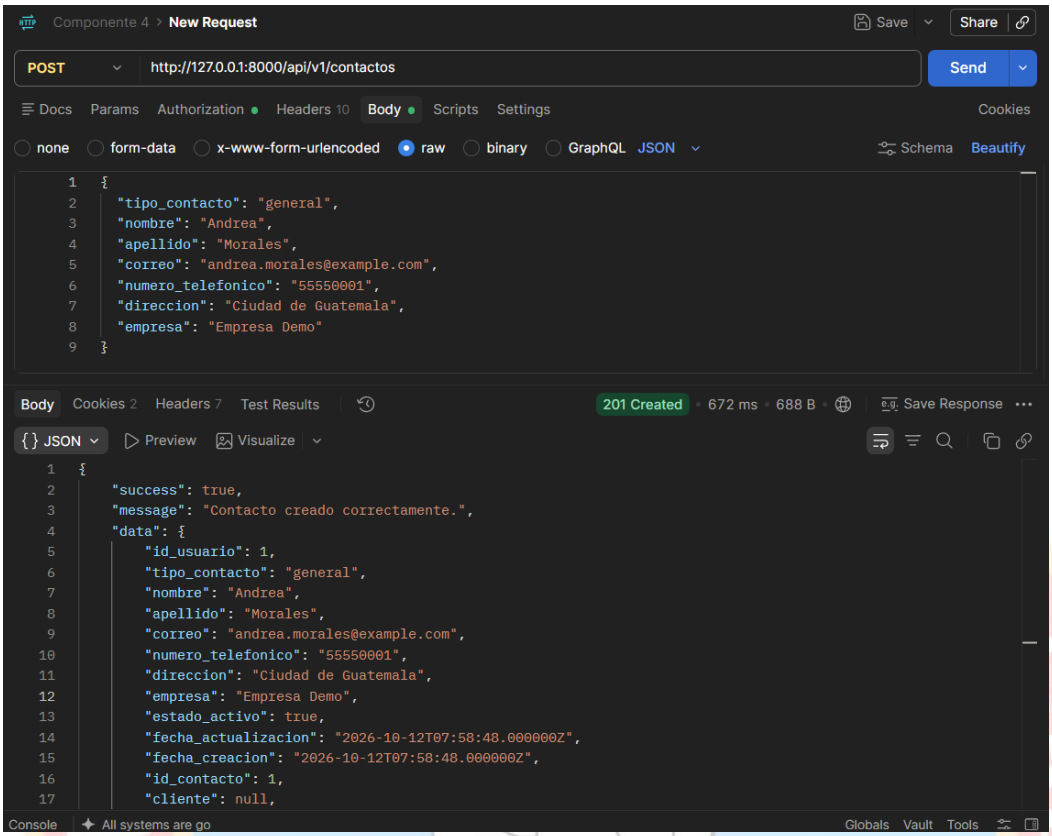
Coleccio-PostMan

APLICACIÓN BACKEN FUNCIONAL Y ENPOINTS EJECUTABLES

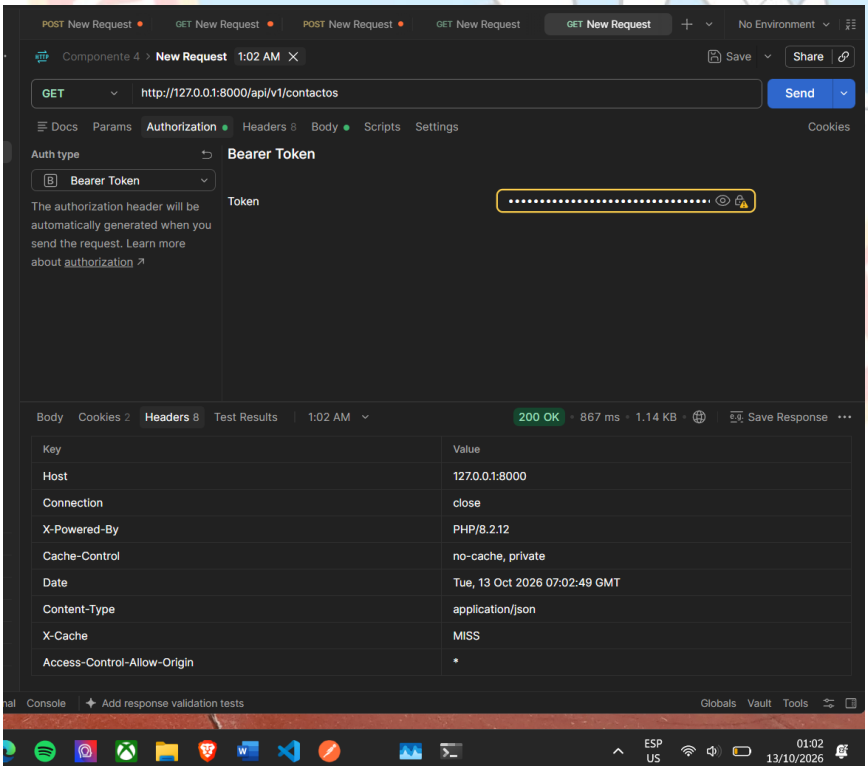



CONOCEREIS LA VERDAD
Y LA VERDAD OS HARA LIBRES
GUATEMALA

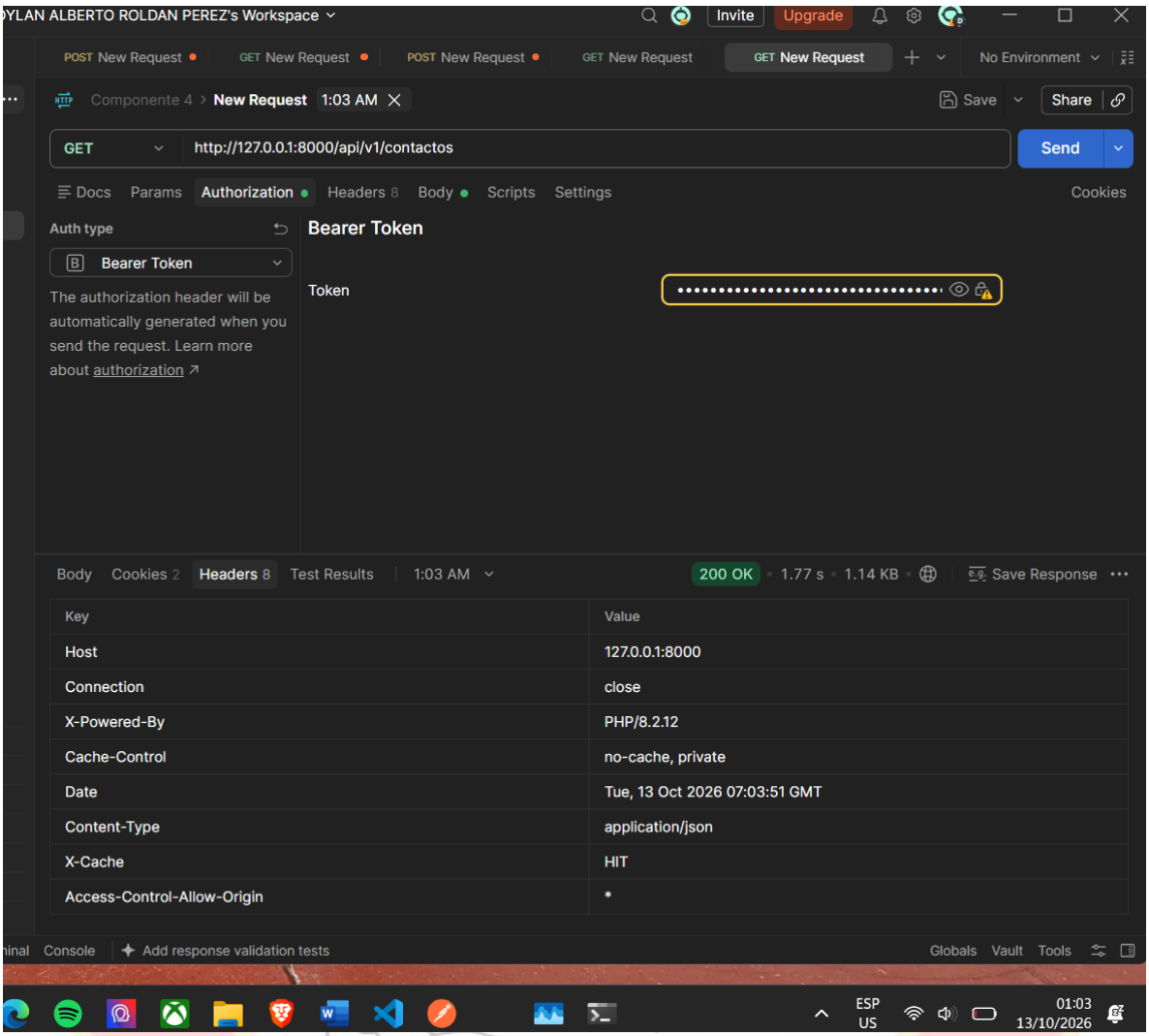

Componente 4 > New Request
Save
Share
GET
http://127.0.0.1:8000/api/v1/contactos
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
Auth type
Bearer Token
The authorization header will be automatically generated when you send the request. Learn more about authorization
Token
Body
Cookies 2
Headers 7
Test Results
200 OK
596 ms
369 B
Save Response
JSON
Preview
Visualize
{
"success": true,
"message": "Contactos consultados correctamente.",
"data": [],
"meta": {
"pagina_actual": 1,
"ultima_pagina": 1,
"por_pagina": 10,
"total": 0
}
}




PRUEBA DE CACHE - CACHEMISS



PRUEBA CACHE – HIT




CONOCEREIS LA VERDAD
Y LA VERDAD OS HARA LIBRES
GUATEMALA

# 5. EXPERIENCIA DE USUARIO (UX/UI) Y MOBILE-FIRST

## 5.1 Diseño Centrado en el Usuario (ISO 9241-210) y Customer Journey Map

El diseño de la interfaz del Módulo CRM se desarrolla siguiendo los principios de diseño centrado en el usuario establecidos por ISO 9241-210. El objetivo es que las funciones principales del sistema sean comprensibles, eficientes y fáciles de utilizar para los usuarios encargados de gestionar contactos.

El proceso considera el contexto de uso, las necesidades de los usuarios, el diseño de soluciones y la evaluación de las interfaces. Para el CRM se identifican principalmente dos perfiles: el usuario estándar, encargado de consultar y gestionar sus contactos, y el administrador, que posee permisos adicionales de supervisión y administración.

La interfaz prioriza las tareas frecuentes: autenticarse, consultar contactos, buscar información, registrar nuevos contactos y visualizar sus detalles. El diseño se plantea de forma iterativa, permitiendo realizar mejoras a partir de evaluaciones de usabilidad.

### Customer Journey Map

| Etapa | Acción del usuario | Necesidad | Posible dificultad | Solución propuesta |
|---|---|---|---|---|
| Acceso | Inicia sesión | Entrar al CRM de forma segura | Credenciales incorrectas | Mensajes claros y formulario simple |
| Consulta | Visualiza contactos | Encontrar información rápidamente | Muchos registros | Búsqueda, filtros y paginación |
| Búsqueda | Filtra por nombre, tipo o estado | Localizar un contacto específico | Resultados difíciles de distinguir | Filtros visibles y resultados ordenados |
| Registro | Crea un contacto | Guardar información correctamente | Campos incompletos o inválidos | Validación inmediata y mensajes de ayuda |
| Detalle | Consulta un contacto | Revisar información completa | Información desorganizada | Agrupar datos generales y específicos |
| Confirmación | Finaliza una operación | Saber si fue exitosa | No saber si se guardaron los cambios | Mensajes claros de éxito o error |

El Customer Journey Map permitió identificar los puntos donde el usuario puede experimentar confusión o retrasos. A partir de estos hallazgos se priorizaron formularios simples, mensajes de retroalimentación, filtros visibles y una navegación consistente.

## 5.2 Evaluación Heurística, ISO 9241-11 y WCAG 2.2

La interfaz del Módulo CRM se evalúa considerando las heurísticas de usabilidad de Nielsen y los principios de usabilidad definidos por ISO 9241-11. El objetivo es verificar que las tareas principales puedan realizarse de forma efectiva, eficiente y satisfactoria.

La evaluación considera los siguientes aspectos:

| **Heurística / criterio** | **Aplicación en el CRM** |
|---|---|
| Visibilidad del estado del sistema | Mostrar mensajes de carga, éxito, error y confirmación de operaciones. |
| Correspondencia con el mundo real | Utilizar términos comprensibles como contactos, clientes, proveedores y estado. |
| Control y libertad del usuario | Permitir cancelar acciones o regresar sin perder información innecesariamente. |
| Consistencia y estándares | Mantener botones, formularios, colores y navegación con comportamiento uniforme. |
| Prevención de errores | Validar campos antes de enviar formularios. |
| Reconocimiento antes que recuerdo | Mantener acciones principales visibles y filtros accesibles. |
| Flexibilidad y eficiencia | Incorporar búsqueda, filtros y paginación. |
| Diseño minimalista | Mostrar únicamente información relevante en cada pantalla. |
| Recuperación ante errores | Presentar mensajes claros indicando qué ocurrió y cómo corregirlo. |
| Ayuda y documentación | Incluir etiquetas, placeholders y mensajes de apoyo en formularios. |

En relación con ISO 9241-11, el sistema busca mejorar tres características principales: **efectividad**, permitiendo completar correctamente las tareas; **eficiencia**, reduciendo pasos innecesarios; y **satisfacción**, proporcionando una interacción clara y predecible.

**Accesibilidad WCAG 2.2 – Principios POUR**

La interfaz se diseña considerando un nivel objetivo **AA de WCAG 2.2**, aplicando los principios POUR:

| Principio | Aplicación propuesta |
|---|---|
| **Perceptible** | Contraste adecuado, etiquetas visibles, textos legibles y alternativas textuales cuando corresponda. |
| **Operable** | Navegación mediante teclado, controles accesibles y áreas táctiles suficientemente grandes. |
| **Comprensible** | Lenguaje claro, navegación consistente y mensajes de error específicos. |
| **Robusto** | Uso correcto de HTML semántico y componentes compatibles con tecnologías de asistencia. |

También se consideran prácticas como mantener contraste suficiente entre texto y fondo, asociar correctamente las etiquetas con los campos de formulario, evitar depender únicamente del color para comunicar estados y conservar un orden lógico de navegación con teclado.

## 5.3 Estrategia Mobile-First y Atomic Design

El diseño del Módulo CRM se plantea bajo una estrategia **Mobile-First**, comenzando por interfaces optimizadas para dispositivos móviles y ampliándolas progresivamente para tabletas y equipos de escritorio.

Este enfoque permite priorizar el contenido esencial y las tareas más importantes, como autenticarse, consultar contactos, buscar información, registrar un contacto y revisar sus detalles.

En dispositivos móviles se prioriza:

- Navegación simple.
- Formularios en una sola columna.
- Botones de fácil acceso táctil.
- Contenido prioritario visible sin saturar la pantalla.
- Tablas adaptadas a tarjetas o vistas resumidas.
- Filtros desplegables cuando el espacio es reducido.
- Diseño responsivo mediante puntos de ruptura.

Para organizar los componentes visuales se utiliza la metodología **Atomic Design**:

| Nivel | Aplicación en el CRM |
|---|---|
| **Átomos** | Botones, inputs, labels, iconos, colores y tipografía. |
| **Moléculas** | Campo de formulario con label + input + mensaje de validación, barra de búsqueda, selector de tipo. |

| Nivel | Aplicación en el CRM |
|---|---|
| **Organismos** | Formulario de contacto, listado de contactos, barra de navegación, panel de filtros. |
| **Plantillas** | Estructura general de Login, Listado, Registro y Detalle. |
| **Páginas** | Pantallas completas con datos reales o simulados del CRM. |

La combinación de Mobile-First y Atomic Design permite reutilizar componentes, mantener consistencia visual y facilitar la evolución de la interfaz.

Vínculo al Prototipo Interactivo:

LINK

# 6. ESTRATEGIA DE CALIDAD Y PLAN SQA

## 6.1 Plan de Aseguramiento de Calidad SQA (IEEE 730-2014) y Enfoque Shift-Left

El Plan de Aseguramiento de Calidad del Módulo CRM se establece con base en IEEE 730-2014, con el propósito de definir actividades, responsabilidades, controles y evidencias que permitan verificar la calidad del producto durante todo el ciclo de desarrollo.

La estrategia SQA no se limita a probar el sistema al finalizar su implementación. Se adopta un enfoque **Shift-Left**, incorporando actividades de calidad desde las primeras fases del proyecto: revisión de requerimientos, validación del contrato OpenAPI, revisión arquitectónica, pruebas unitarias, pruebas de integración, escenarios BDD y pruebas funcionales de los endpoints.

De esta manera, los errores pueden ser detectados antes de llegar a etapas avanzadas, reduciendo el costo de corrección y aumentando la trazabilidad entre requerimientos, diseño, implementación y pruebas.

| Actividad SQA | Evidencia |
|---|---|
| Revisión de requerimientos | SRS ISO 29148 |
| Revisión de arquitectura | ADR, C4 e ISO 25010 |
| Validación del contrato API | openapi.yaml |
| Validación BDD | Archivos .feature |
| Pruebas funcionales | Postman |
| Pruebas de caché | Cache MISS / HIT / invalidación |
| Pruebas de persistencia | PostgreSQL y Redis |
| Gestión de defectos | Reporte IEEE 1044 |
| Trazabilidad | Matriz integral |

**QA vs QC**

**QA (Quality Assurance)** se enfoca en prevenir defectos mediante procesos, estándares, revisiones y buenas prácticas.

**QC (Quality Control)** se enfoca en detectar defectos mediante pruebas y verificación del producto construido.

## 6.2 Pirámide de Pruebas Automatizadas (Unitarias, Integración, E2E)

La estrategia de pruebas del Módulo CRM se organiza mediante una pirámide de pruebas, priorizando una mayor cantidad de pruebas rápidas y aisladas en la base, seguidas de pruebas de integración y un número menor de pruebas End-to-End.

Este enfoque permite detectar defectos con rapidez, reducir el costo de mantenimiento y validar progresivamente la interacción entre los distintos componentes del sistema.

| Nivel | Objetivo | Ejemplos en el CRM |
|---|---|---|
| **Unitarias** | Validar funciones o clases de forma aislada | DTOs, validaciones, servicios, reglas de negocio |
| **Integración** | Comprobar la interacción entre componentes | Controller → Service → Repository → PostgreSQL / Redis |
| **E2E / Funcionales** | Validar flujos completos desde el consumidor | Login, listar contactos, crear contacto y consultar detalle mediante Postman |

La mayor cantidad de pruebas debe concentrarse en el nivel unitario, ya que son más rápidas y fáciles de mantener. Las pruebas de integración validan la comunicación entre módulos y persistencias, mientras que las pruebas E2E verifican los flujos funcionales completos del sistema.

En el proyecto, las pruebas funcionales realizadas mediante Postman permiten comprobar autenticación JWT, persistencia en PostgreSQL, uso de Redis mediante Cache-Aside y respuestas HTTP de los endpoints implementados.

## 6.3 Clasificación y Flujo de Gestión de Defectos (IEEE 1044-2009)

La gestión de defectos del Módulo CRM se define tomando como referencia IEEE 1044-2009. Cada defecto detectado debe registrarse, clasificarse, priorizarse y dar seguimiento hasta su resolución y verificación.

El objetivo es mantener un control formal de los errores encontrados durante pruebas unitarias, integración, pruebas funcionales y validaciones del sistema.

| Severidad | Descripción | Ejemplo en el CRM |
|---|---|---|
| **Crítica** | Impide utilizar una función esencial o compromete seguridad/datos | No se puede iniciar sesión o se pierden contactos |
| **Alta** | Función importante falla sin solución alternativa aceptable | No se pueden crear contactos |
| **Media** | El sistema funciona, pero con comportamiento incorrecto parcial | Filtro devuelve resultados incorrectos |
| **Baja** | Problema menor sin impacto importante en la operación | Texto mal alineado o mensaje poco claro |

| Severidad Descripción | Ejemplo en el CRM |
|---|---|
| **Prioridad** | **Tratamiento** |
| **P1 - Urgente** | Corregir inmediatamente |
| **P2 - Alta** | Corregir antes de la siguiente entrega |
| **P3 - Media** | Programar en el ciclo actual o próximo |
| **P4 - Baja** | Puede esperar sin afectar el funcionamiento |

## 7. MATRIZ DE TRAZABILIDAD INTEGRAL

La matriz de trazabilidad permite demostrar la correspondencia entre los requerimientos definidos bajo ISO/IEC/IEEE 29148, las Historias de Usuario, la arquitectura, el contrato OpenAPI, la implementación backend, los escenarios BDD y los casos de prueba. Su objetivo es garantizar que cada requerimiento tenga evidencia de diseño, implementación y verificación.

| Requerimiento | HU | C4 / Componente | Endpoint OpenAPI | Código Backend | BDD | Caso de prueba |
|---|---|---|---|---|---|---|
| REQ-FN-001 Autenticar usuario | HU-01 | Auth / Servicio de autenticación | POST /api/v1/auth/login | src/app/Http/Controllers/Api/V1/AuthController.php | autenticacion.feature | CP-01, CP-02 |
| REQ-FN-002 Registrar contacto | HU-02 | Contactos / ContactoService | POST /api/v1/contactos | ContactoController@store | gestion_recursos.feature | CP-04, CP-05 |
| REQ-FN-003 Consultar contactos | HU-03 | Contactos / Repository | GET /api/v1/contactos | ContactoController@index | gestion_recursos.feature | CP-03 |
| REQ-FN-003 Consultar detalle | HU-03 | Contactos / Repository | GET /api/v1/contactos/{id} | ContactoController@show | gestion_recursos.feature | Caso funcional Postman |
| REQ-NF-006 Aplicar Cache-Aside | HU-03 | Redis / Cache Service | GET /api/v1/contactos | ContactoController@index | Escenario Cache Hit/Miss | CP-08 |

| Requerimiento | HU | C4 / Componente | Endpoint OpenAPI | Código Backend | BDD | Caso de prueba |
|---|---|---|---|---|---|---|
| REQ-NF-007 Continuar si Redis falla | HU-03 | PostgreSQL / Redis | GET /api/v1/contactos | Servicio / Repository | Escenario fallback | CP-09 |
| REQ-NF-004 Estandarizar errores | Transversal | API | Todos los endpoints | Controladores API | Escenarios de error | CP-02, CP-05 |
| REQ-NF-008 Contrato OpenAPI | Transversal | API | openapi.yaml | Rutas API | Features BDD | Validación del contrato |

La trazabilidad evidencia el enfoque Shift-Left aplicado al proyecto, ya que los requerimientos definidos al inicio se relacionan con el contrato API, la arquitectura, la implementación y las pruebas. Esto permite identificar rápidamente qué elementos deben revisarse cuando un requerimiento cambia o cuando se detecta un defecto.

VINCULOS

plan_sqa_ieee730.pdf

reporte_defectos_ieee1044.pdf

# PRUEBAS

```php
<?php

namespace Tests\Feature;

use Tests\TestCase;

class ExampleTest extends TestCase
{
    public function test_guest_is_redirected_to_login(): void
    {
        $response = $this->get('/');

        $response->assertRedirect('/login');
    }
}
```

```
Output  Debug Console  Terminal  Ports  Problems

PS C:\xampp\htdocs\modulocrm\src> php artisan test
  FAILED  Tests\Feature\ExampleTest > the application returns a successful response
  Expected response status code [200] but received 302.
Failed asserting that 302 is identical to 200.

  at tests\Feature\ExampleTest.php:17
     13      public function test_the_application_returns_a_successful_response(): void
     14      {
     15          $response = $this->get('/');
     16
  →  17          $response->assertStatus(200);
     18      }
     19  }
     20

  Tests:    1 failed, 1 passed (2 assertions)
  Duration: 16.97s

PS C:\xampp\htdocs\modulocrm\src> php artisan test

  PASS  Tests\Unit\ExampleTest
  ✓ that true is true                                                    0.01s

  PASS  Tests\Feature\ExampleTest
  ✓ guest is redirected to login                                         0.43s

  Tests:    2 passed (3 assertions)
  Duration: 0.82s

PS C:\xampp\htdocs\modulocrm\src>
```

CONOCEREIS LA VERDAD
Y LA VERDAD OS HARA LIBRES
GUATEMALA

```php
class ContactoServiceTest extends TestCase
    public function test_obtener_por_id_retorna_null_si_el_contacto_no_existe(): void

        $repositorio
            ->shouldReceive('obtenerPorId')
            ->once()
            ->with(999)
            ->andReturn(null);

        $servicio = new ContactoService($repositorio);

        // Act
        $resultado = $servicio->obtenerPorId(999);

        // Assert
        $this->assertNull($resultado);
    }
}
```

```
utput  Debug Console  Terminal  Ports  Problems

PS C:\xampp\htdocs\modulocrm\src> php artisan test

  Tests:    1 failed, 1 passed (2 assertions)
  Duration: 16.97s

PS C:\xampp\htdocs\modulocrm\src> php artisan test

  PASS  Tests\Unit\ExampleTest
  ✓ that true is true                                                    0.01s

  PASS  Tests\Feature\ExampleTest
  ✓ guest is redirected to login                                         0.43s

  Tests:    2 passed (3 assertions)
  Duration: 0.82s

PS C:\xampp\htdocs\modulocrm\src> php artisan make:test ContactoServiceTest --unit

  INFO  Test [C:\xampp\htdocs\modulocrm\src\tests\Unit\ContactoServiceTest.php] created successfully.

PS C:\xampp\htdocs\modulocrm\src> php artisan test --filter=ContactoServiceTest

  PASS  Tests\Unit\ContactoServiceTest
  ✓ obtener por id retorna el contacto del repositorio                   1.91s
  ✓ obtener por id retorna null si el contacto no existe                 0.06s

  Tests:    2 passed (4 assertions)
  Duration: 2.28s

PS C:\xampp\htdocs\modulocrm\src>
```

### Prueba 1 – Obtener un contacto existente:

Verifica que ContactoService solicite correctamente un contacto al repositorio mediante su ID y devuelva un objeto Contacto con los datos esperados. Para aislar la prueba, se utiliza un repositorio simulado con Mockery.

### Prueba 2 – Obtener un contacto inexistente:

Comprueba que ContactoService devuelva null cuando el repositorio no encuentra un contacto con el ID solicitado. También utiliza un repositorio simulado, por lo que no depende de PostgreSQL ni de Redis.

En conjunto, ambas pruebas validan el comportamiento del método obtenerPorId() de forma aislada, por eso pertenecen al nivel de **pruebas unitarias** de la pirámide.

```
Output  Debug Console  Terminal  Ports  Problems

PS C:\xampp\htdocs\modulocrm\src> php artisan config:clear

  INFO  Configuration cache cleared successfully.

PS C:\xampp\htdocs\modulocrm\src> php artisan test --filter=AuthApiTest

  PASS  Tests\Feature\AuthApiTest
  ✓ usuario activo puede iniciar sesion y recibir jwt                    20.3
  ✓ login con password incorrecto retorna 401                             0.1

  Tests:    2 passed (9 assertions)
  Duration: 21.03s

PS C:\xampp\htdocs\modulocrm\src>
```

```
PS C:\xampp\htdocs\modulocrm\src> php artisan make:test ContactosApiTest

  INFO  Test [C:\xampp\htdocs\modulocrm\src\tests\Feature\ContactosApiTest.php] created successfully.

PS C:\xampp\htdocs\modulocrm\src> docker exec crm-redis redis-cli ping
PONG
PS C:\xampp\htdocs\modulocrm\src> php artisan test --filter=ContactosApiTest

  PASS  Tests\Feature\ContactosApiTest
  ✓ endpoint contactos requiere autenticacion
  ✓ listado contactos utiliza postgresql y cache redis
  ✓ contacto inexistente retorna problem details 404

  Tests:    3 passed (16 assertions)
  Duration: 6.30s

PS C:\xampp\htdocs\modulocrm\src>
```

```
PS C:\xampp\htdocs\modulocrm\src> php artisan test

  PASS  Tests\Unit\ContactoServiceTest
  ✓ obtener por id retorna el contacto del repositorio
  ✓ obtener por id retorna null si el contacto no existe

  PASS  Tests\Unit\ExampleTest
  ✓ that true is true

  PASS  Tests\Feature\AuthApiTest
  ✓ usuario activo puede iniciar sesion y recibir jwt
  ✓ login con password incorrecto retorna 401

  PASS  Tests\Feature\ContactosApiTest
  ✓ endpoint contactos requiere autenticacion
  ✓ listado contactos utiliza postgresql y cache redis
  ✓ contacto inexistente retorna problem details 404

  PASS  Tests\Feature\ExampleTest
  ✓ guest is redirected to login

  Tests:    9 passed (32 assertions)
  Duration: 5.94s

PS C:\xampp\htdocs\modulocrm\src>
```

| Prueba | Descripción |
|---|---|
| **ContactoServiceTest – contacto existente** | Verifica que ContactoService solicite correctamente un contacto al repositorio y devuelva el objeto esperado cuando el ID existe. |
| **ContactoServiceTest – contacto inexistente** | Comprueba que el servicio devuelva null cuando el repositorio no encuentra un contacto con el ID solicitado. |
| **AuthApiTest – login válido** | Valida que un usuario activo pueda autenticarse con credenciales correctas y reciba un token JWT junto con una respuesta HTTP 200. |
| **AuthApiTest – contraseña incorrecta** | Comprueba que una contraseña inválida sea rechazada y que la API responda con HTTP 401. |
| **ContactosApiTest – endpoint protegido** | Verifica que GET /api/v1/contactos no pueda ser utilizado sin autenticación JWT y responda con HTTP 401. |
| **ContactosApiTest – PostgreSQL y Redis** | Comprueba el patrón Cache-Aside: la primera consulta produce X-Cache: MISS y obtiene los datos desde PostgreSQL; la segunda produce X-Cache: HIT utilizando Redis. |
| **ContactosApiTest – contacto inexistente** | Valida que al consultar un contacto inexistente la API responda con HTTP 404 y un error estructurado con application/problem+json. |

# 8. CONCLUSIONES

- El principal problema del sistema CRM original era su fuerte dependencia entre la lógica de negocio, acceso a datos y controladores, lo que dificultaba el mantenimiento y la evolución del sistema. La reorganización hacia un Monolito Modular, separando responsabilidades mediante Controller, Service y Repository, permite una estructura más clara y fácil de mantener.

- La arquitectura propuesta mejora atributos de calidad como mantenibilidad, rendimiento, seguridad y escalabilidad. PostgreSQL funciona como fuente principal de datos, mientras que Redis implementa el patrón Cache-Aside para reducir consultas repetitivas. Además, la autenticación mediante JWT permite proteger los endpoints de la API.

- La aplicación de un enfoque API-First, utilizando OpenAPI 3.0.3, BDD con Gherkin y pruebas automatizadas permite mantener una mayor correspondencia entre requerimientos, contrato, implementación y pruebas. Esto facilita detectar inconsistencias antes de que lleguen a etapas avanzadas del desarrollo.

- Las pruebas realizadas permitieron comprobar funcionalmente la autenticación, consulta y registro de contactos, respuestas HTTP, persistencia en PostgreSQL y funcionamiento de Redis mediante escenarios de MISS y HIT. También se incorporaron pruebas unitarias y de integración automatizadas, reforzando la estrategia de calidad basada en Shift-Left.

- Aunque la solución propuesta contempla una arquitectura de persistencia políglota con PostgreSQL, Redis, MongoDB y Neo4j, actualmente PostgreSQL y Redis son los componentes integrados funcionalmente. MongoDB y Neo4j permanecen como parte de la evolución arquitectónica prevista para historial y relaciones. Antes de una implementación en producción también sería necesario fortalecer monitoreo, manejo global de errores, seguridad, respaldo de datos, disponibilidad de Redis y automatización de despliegues.

# 9. REFERENCIAS


- International Organization for Standardization. (2018). *ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering*. ISO. ISO
- Institute of Electrical and Electronics Engineers. (2009). *IEEE 1016-2009 — IEEE Standard for Information Technology—Systems Design—Software Design Descriptions*. IEEE Standards Association. IEEE Standards Association
- International Organization for Standardization. (2023). *ISO/IEC 25010:2023 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. ISO. ISO
- International Organization for Standardization. (2022). *ISO/IEC/IEEE 42010:2022 — Software, systems and enterprise — Architecture description*. ISO. ISO
- OpenAPI Initiative. (2020). *OpenAPI Specification Version 3.0.3*. OpenAPI Initiative Publications
- Fielding, R., Nottingham, M., & Reschke, J. (2022). *RFC 9110: HTTP Semantics*. Internet Engineering Task Force. RFC Editor
- Nottingham, M., Wilde, E., & Dalal, S. (2023). *RFC 9457: Problem Details for HTTP APIs*. Internet Engineering Task Force. RFC Editor
- International Organization for Standardization. (2021). *ISO/IEC/IEEE 29119-3:2021 — Software and systems engineering — Software testing — Part 3: Test documentation*. ISO. ISO
- Institute of Electrical and Electronics Engineers. (2014). *IEEE 730-2014 — IEEE Standard for Software Quality Assurance Processes*. IEEE Standards Association. IEEE Standards Association
- Institute of Electrical and Electronics Engineers. (2009). *IEEE 1044-2009 — IEEE Standard Classification for Software Anomalies*. IEEE Standards Association. IEEE Standards Association
- International Organization for Standardization. (2019). *ISO 9241-210:2019 — Ergonomics of human-system interaction — Part 210: Human-centred design for interactive systems*. ISO. ISO
- International Organization for Standardization. (2018). *ISO 9241-11:2018 — Ergonomics of human-system interaction — Part 11: Usability: Definitions and concepts*. ISO. ISO
- World Wide Web Consortium. (2024). *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C. W3C
- Brown, S. (s. f.). *The C4 Model for visualising software architecture*. C4 model
- Frost, B. (s. f.). *Atomic Design Methodology*.


UNIVERSIDAD MARIANO GALVEZ
1966
CONOCEREIS LA VERDAD
Y LA VERDAD OS HARA LIBRES
GUATEMALA